Ex. No : 16

Date : Data Abstraction in Python

Aim

To understand the concept of Data Abstraction in Python using abstract classes and methods.

Theory

Data Abstraction is the process of hiding implementation details and showing only the essential features of an object. In Python, abstraction is achieved using abstract classes and abstract methods with the help of the abc (Abstract Base Class) module.

16.1 Program Demonstrating Abstract Class and Abstract Method

Program

```
from abc import ABC, abstractmethod

class Animal(ABC):

    @abstractmethod
    def sound(self):
        pass

class Dog(Animal):

    def sound(self):
        print("Dog barks")

obj = Dog()
obj.sound()
```

Sample Output

```
Dog barks
```

16.2 Program Demonstrating Abstraction Using Shapes

Program

```
from abc import ABC, abstractmethod

class Shape(ABC):

    @abstractmethod
    def area(self):
        pass

class Rectangle(Shape):

    def __init__(self, length, breadth):
        self.length = length
        self.breadth = breadth

    def area(self):
        print("Area =", self.length * self.breadth)

obj = Rectangle(10, 5)
obj.area()
```

Sample Output

```
Area = 50
```

16.3 Program Demonstrating Multiple Abstract Methods

Program

```
from abc import ABC, abstractmethod

class Vehicle(ABC):

    @abstractmethod
    def start(self):
        pass

    @abstractmethod
    def stop(self):
```

```
        pass

class Car(Vehicle):

    def start(self):
        print("Car started")

    def stop(self):
        print("Car stopped")

obj = Car()
obj.start()
obj.stop()
```

Sample Output

```
Car started
Car stopped
```

Result

The programs demonstrating Data Abstraction in Python were successfully executed.

Viva Questions

1. What is data abstraction?
2. What is an abstract class?
3. What is an abstract method?
4. Which module is used for abstraction in Python?
5. Why is abstraction important in programming?

Exercises

1. Create an abstract class for employees.
2. Write a program to demonstrate abstraction in banking applications.

\newpage

Ex. No : 17

Date : Polymorphism in Python

Aim

To understand the concept of Polymorphism in Python.

Theory

Polymorphism means “many forms”. It allows the same function, method, or operator to behave differently depending on the object or data type. Polymorphism improves flexibility and code reusability in object-oriented programming.

17.1 Program Demonstrating Method Overriding

Program

```
class Animal:

    def sound(self):
        print("Animal makes sound")

class Dog(Animal):

    def sound(self):
        print("Dog barks")

obj = Dog()
obj.sound()
```

Sample Output

```
Dog barks
```

17.2 Program Demonstrating Polymorphism with Functions

Program

```
class Cat:

    def sound(self):
        print("Cat meows")

class Cow:

    def sound(self):
        print("Cow moos")

for animal in (Cat(), Cow()):
    animal.sound()
```

Sample Output

```
Cat meows
Cow moos
```

17.3 Program Demonstrating Operator Overloading

Program

```
class Addition:

    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return self.value + other.value

obj1 = Addition(10)
obj2 = Addition(20)

print("Addition =", obj1 + obj2)
```

Sample Output

```
Addition = 30
```

17.4 Program Demonstrating Method Overloading Using Default Arguments

Program

```
class Display:

    def show(self, a=None, b=None):
        if a is not None and b is not None:
            print("Sum =", a + b)
        elif a is not None:
            print("Value =", a)
        else:
            print("No arguments")

obj = Display()
obj.show()
obj.show(10)
obj.show(10, 20)
```

Sample Output

```
No arguments
Value = 10
Sum = 30
```

Result

The programs demonstrating Polymorphism in Python were successfully executed.

Viva Questions

1. What is polymorphism?
2. What is method overriding?

3. What is operator overloading?
4. What is the difference between overloading and overriding?
5. How does polymorphism improve programming?

Exercises

1. Write a program to demonstrate polymorphism using shapes.
2. Create a class hierarchy to demonstrate method overriding.